

TASK 13

1. ATS SCORING ENGINE & Configurable Weight System

ats_engine/ats_scorer.py

```
DEFAULT_WEIGHTS = {
```

```
    "skills": 0.35,
```

```
    "experience": 0.25,
```

```
    "education": 0.15,
```

```
    "semantic": 0.25
```

```
}
```

```
ROLE_WEIGHTS = {
```

```
    "backend developer": {
```

```
        "skills": 0.40,
```

```
        "experience": 0.30,
```

```
        "education": 0.10,
```

```
        "semantic": 0.20
```

```
    },
```

```
    "data scientist": {
```

```
        "skills": 0.35,
```

```
        "experience": 0.25,
```

```
        "education": 0.20,
```

```
        "semantic": 0.20
```

```
    },
```

```
    "marketing executive": {
```

```
        "skills": 0.30,
```

```
        "experience": 0.30,
```

```
        "education": 0.15,
```

```

        "semantic": 0.25
    }}

def normalize(score):
    return score / 100 if score else 0

def get_weights(job_role):
    return ROLE_WEIGHTS.get(
        job_role.lower(),
        DEFAULT_WEIGHTS
    )

def calculate_at_score(
    candidate,
    job,
    weights=DEFAULT_WEIGHTS
):
    skill_score = normalize(
        candidate.get("skill_score", 0)
    )
    exp_score = normalize(
        candidate.get("experience_score", 0)
    )
    edu_score = normalize(
        candidate.get("education_score", 0)
    )
    semantic_score = normalize(
        candidate.get("semantic_score", 0)
    )
    final_score = (
        weights["skills"] * skill_score +

```

```

        weights["experience"] * exp_score +
        weights["education"] * edu_score +
        weights["semantic"] * semantic_score
    )
    return round(final_score * 100, 2)
def generate_candidate_score(candidate, job):
    weights = get_weights(
        job.get("job_title", "")
    )
    final_score = calculate_ats_score(
        candidate,
        job,
        weights
    )
    return {
        "candidate_id": candidate.get(
            "candidate_id"
        ),
        "final_score": final_score,
        "weights_used": weights,
        "breakdown": {
            "skill_score": candidate.get(
                "skill_score",
                0
            ),
            "experience_score": candidate.get(
                "experience_score",
                0
            )
        }
    }

```

```

    ),
    "education_score": candidate.get(
        "education_score",
        0
    ),
    "semantic_score": candidate.get(
        "semantic_score",
        0
    )
}}

def safe_score(value):
    return value if value is not None else 0

def calculate_safe_ats(candidate, job):
    candidate["skill_score"] = safe_score(
        candidate.get("skill_score")
    )
    candidate["experience_score"] = safe_score(
        candidate.get("experience_score")
    )
    candidate["education_score"] = safe_score(
        candidate.get("education_score")
    )
    candidate["semantic_score"] = safe_score(
        candidate.get("semantic_score")
    )
    return generate_candidate_score(
        candidate,
        job
    )

```

3. Candidate Score Generator

main.py

```
from ats_engine.ats_scorer import (
    generate_candidate_score
)

# ♦ Step 5: ATS Scoring Engine
candidate_scores = {
    "candidate_id": "C123",
    "skill_score": 80,
    "experience_score": 75,
    "education_score": 70,
    "semantic_score": semantic_result[
        "final_similarity_score"
    ]
}

ats_result = generate_candidate_score(
    candidate_scores,
    job
)

print("\nATS SCORE RESULT:\n")

print(
    json.dumps(
        ats_result,
        indent=2
    )
)
```

OUTPUT

ATS SCORE RESULT:

```
{  
  "candidate_id": "C123",  
  "final_score": 74.12,  
  "weights_used": {  
    "skills": 0.35,  
    "experience": 0.25,  
    "education": 0.15,  
    "semantic": 0.25  
  },  
  "breakdown": {  
    "skill_score": 80,  
    "experience_score": 75,  
    "education_score": 70,  
    "semantic_score": 67.48  
  }  
}
```